

- Raspberry pi Serial Console 설정

Serial console 설정

```
~$ sudo vi /boot/firmware/config.txt  #이름 아래 추가
```

```
enable_uart=1
```

```
~$ sudo vi /boot/firmware/cmdline.txt
```

```
// PARTUUID 기존 값으로 사용
```

```
console=serial0,115200
```

```
console=ttyS0
```

```
root=PARTUUID=36f9da29-
```

```
02 rootfstype=ext4 fsck.repair=yes rootwait
```

```
usb to serial 케이블 연결
```



6번(GND) → usb 케이블 녹색(GND)

8번(TXD) → usb 케이블 흰색(RXD)

10번(RXD) → usb 케이블 초록색(TXD)

3.2 U-Boot Bootloader 펌드(ubuntu 호스트 크로스 컴파일)

```
~$ mkdir pi_bsp : cd pi_bsp //디렉토리 생성 및 이동
```

필수 패키지 설치

```
$ sudo apt update
```

```
$ sudo apt-get install gawk git-core diffstat unzip texinfo gcc-multilib build-essential chrpath socat cpio python-setuptools python3-pip python3-pexpect xz-utils debianutils iputils-ping python3-gi python3-jinja2 libegl1-mesa libsdl1.2-dev xterm rsync curl zstd lz4 bison flex libssl-dev libgnutls28-dev
```

```
~/pi_bsp$ sudo apt install crossbuild-essential-armhf //arm 32bit 플랫폼인 설치
```

```
~/pi_bsp$ git clone git://git.denx.de/u-boot.git //u-boot 다운로드
```

```
~/pi_bsp$ cd u-boot
```

```
$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-rpi_4-32b_defconfig
```

```
$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-all
```

```
~$ u-boot 타겟보드에 적제 하기 1
```

```
라즈베리파이 SD카드 ubuntu 연결 후 (가장머신 usb3.0 플성화)
```

```
$ df
```

```
/dev/sdb1 261108 83214 177894 32% /media/ubuntu/boots
```

```
u-boot$ cp u-boot.bin /media/ubuntu/boots/ //u-boot.bin sd카드 복사
```

```
~$ u-boot 타겟보드에 적제 하기 2
```

```
u-boot.bin 파일을 라즈베리파이 전송(NFS 또는 SCP 명령 사용)
```

```
u-boot$ scp u-boot.bin pi@10.10.141.60:~ //라즈베리 플더렉토리 전송
```

```
pi@pi00:~$ sudo cp u-boot.bin /boot/firmware/ //u-boot.bin 복사
```

```
pi@pi00:~$ sync //sd카드 동기화
```

```
u-boot$ vi /media/ubuntu/boots/config.txt //플라인에 추가, 그래픽모드 현재 안됨
```

```
kernel=u-boot.bin //u-boot 부팅
```

```
라즈베리파이에 sd카드 삽입 후 부팅 확인, 펌드 시간 확인 - 부팅 메시지 첫번째 메시지
```

```
U-Boot 2023.10-rc3-00049-g0fe0395922 (Aug 31 2023 - 11:26:40 +0900)
```

```
U-Boot> setenv bootargs 8250.nr_uarts=1 console=ttyS0,115200
```

```
root=/dev/mmcblk0p2 rootwait rw
```

```
U-Boot> printenv bootcmd //bootcmd 확인
```

```
bootcmd=run distro_bootcmd
```

```
~$ user_mmc_boot U-boot 부팅 스크립트 작성
```

```
U-Boot> setenv bootcmd 'run user_mmc_boot'
```

```
U-Boot> setenv user_mmc_boot 'mmc dev 0; fatload mmc 0:1 ${kernel_addr_r} kernel7.img; fatload mmc 0:1 ${fdt_addr_r} bcm2711-rpi-4-b.dtb; bootz ${kernel_addr_r} - ${fdt_addr_r}'
```

```
U-Boot> saveenv //환경변수 저장
```

```
U-Boot> reset //재부팅
```

```
u-boot 메모리 read/write 테스트 (GPIO6 LED0 On/Off) - 데이터쉬트 확인
```

```
LED 모듈(led0~led7) : gpio6 ~ gpio13, KEY모듈(key0~key7) : gpio16~gpio23
```

```
U-Boot> mdw 0xfe200000 4 //GPFSLE0
```

```
fe200000: 00000000 00012000 12000000 3fff0000 .....?
```

```
U-Boot> mdw 0xfe200000 0x00040000 //GPFSLE0 gpio6 output
```

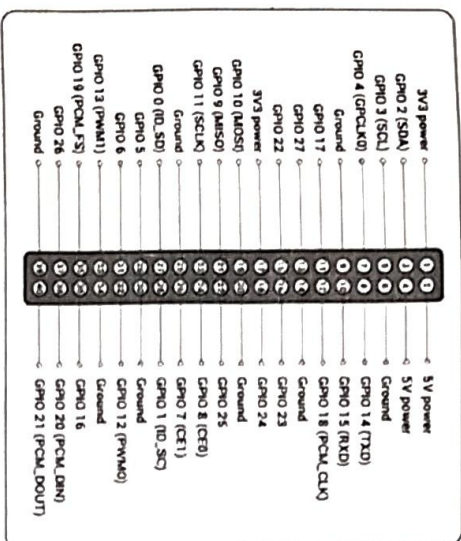
```

U-Boot> mw.l 0xfe20001c 0x00000040 //GPSET0 gpio6 Set(LED0 on)
U-Boot> mw.l 0xfe200028 0x00000040 //GPSET0 gpio6 Clear(LED0 off)

U-Boot> mw.l 0xfe200000 0x09240000 //GPFSEL0 gpio6~9 output
U-Boot> mw.l 0xfe200004 0x00012249 //GPFSEL1 gpio10~13 output
U-Boot> mw.l 0xfe20001c 0x00003fc0 //GPFSEL0 gpio6~13 Set (LED all on)
U-Boot> mw.l 0xfe200028 0x00003fc0 //GPFSEL0 gpio6~13 Clear(LED all off)

```

- 응용실습) 스위치 모듈의 key 테스트



- u-boot 코드 분석 환경

```
~/pi_bsp/u-boot$ sudo apt install vim universal-ctags
```

```
$ARCH=arm make tags
```

```
$vi ~/.vimrc
```

```
set number
```

```
set autoindent
```

```
set cindent
```

```
set smartindent
```

```
set shiftwidth=4
```

```
set ts=4
```

```
set tags+=/home/ubuntu/pi_bsp/u-boot/tags
```

1. ctrl +] => tag 자동 찾기
2. ctrl + t => 이전으로 복귀
3. if 함수명 or 구조체 명 => 지정한 함수 또는 구조체로 jump
4. sis 함수명 or 구조체 명 => if(tag jump) 와 동일하지만 기존화면을 수평으로 split 하여 보여준다. 현재 창과 보고 싶은 tag를 동시에 보이고 할때 유용함.
5. vi -t _start //entry point 파일 오픈

```

- u-boot 분석 및 어셈블리어 실습 - LED0-3 Shift 테스트 어셈블리어 예제
u-boot 시작점 ~/pi_bsp/u-boot$ arch/arm/lib/vectors.S // _start (u-boot.ld)
$vi ~/pi_bsp/u-boot/arch/arm/cpu/armv7/start.S

```

```

127      bl kcci_led_test
128      bl _main

396 ENTRY(kcci_led_test)
397     ldr r0,=0xFE200000 //BCM2711 GPIO BASE*/
398     ldr r1,=0x09240000 //gpio6~9 signal output*/
399     str r1,[r0,#0x00] //GPFSEL0*/
400     ldr r1,=0x00000040 //gpio6 set*/
401
402     mov r2,#4 //led count*/
403 ledloop:
404     str r1,[r0,#0x1C] //GPSET0 ledX on*/
405     ldr r3,=0x400000 //delay 0x400000(4,194,304) / 54MHz = 0.7767Sec */
406 delay:
407     subs r3,r3,#1
408     bne delay
409     str r1,[r0,#0x28] //GPCLR0 ledX off*/
410     mov r1,r1,LSL,#1
411     subs r2,r2,#1
412     bne ledloop
413     mov pc,lr
414 ENDPROC(kcci_led_test)

```

- 빌드 후 sd카드 퓨징 및 부팅시 LED 확인

```
pi@pi00:/mnt/ubuntu_nfs$ sudo cp u-boot.bin /boot/firmware/u-boot.bin
```

- Disassembly 분석

```
~/pi_bsp/u-boot/arch/arm/cpu/armv7$ arm-linux-gnueabi-hf-objdump -S start.o > start.txt
```

```
~/armv7$ vi start.txt 분석 (<?>에어 생성 과정 분석, 데이터 처리 명령어)
```

```

mov r2,#4      => e3a02004
subs r3,r3,#1  => e2533001

```

- 응용실습) 1. 어셈블리어 LED 8개 좌측 및 우측으로 켜프트하기

2. 쉬프트 중 스위치로 종료하기

```

u-boot LED 제어 기능 추가
u-boot cmd 호출 함수 ~pi_bsp/u-boot$ vi common/command.c           //cmd_call

~//u-boot/common$ vi cmd_kcci_led.c
-----
1 #include <vprintf.h>
2 #include <command.h>
3 #include <asm/io.h>
4
5 #define BCM2711_GPIO_GPFSSEL0 0xFE200000
6 #define BCM2711_GPIO_GPFSSEL1 0xFE200004
7 #define BCM2711_GPIO_GPFSSEL2 0xFE200008
8 #define BCM2711_GPIO_GPSET0 0xFE20001C
9 #define BCM2711_GPIO_GPCLR0 0xFE200028
10 #define BCM2711_GPIO_GPLEV0 0xFE200034
11
12 #define GPIO6_9_SIG_OUTPUT BCM2711_GPIO_GPFSSEL0;
13 #define GPIO10_13_SIG_OUTPUT 0x00012249 //vxd,rx
14
15 void led_init(void)
16 {
17     write(GPIO6_9_SIG_OUTPUT, BCM2711_GPIO_GPFSSEL0);
18     write(GPIO10_13_SIG_OUTPUT, BCM2711_GPIO_GPFSSEL1);
19 }
20 void led_write(unsigned long led_data)
21 {
22     write(0x3fc0, BCM2711_GPIO_GPCLR0); //led all off
23     led_data = led_data << 6;
24     write(led_data, BCM2711_GPIO_GPSET0); //ledx on
25 }
26 static int do_KCCLED(struct cmd_tbl *cmdtp, int flag, int argc, char * const argv
[1])
27 {
28     unsigned long led_data;
29     if(argc != 2)

```

```

30 {
31     cmd_usage(cmdtp);
32     return 1;
33 }
34 printf("LED TEST START\n");
35 led_init();
36 led_data = simple_strtol(argv[1], NULL, 16);
37 led_write(led_data);
38 printf("LED TEST END(%s : %04x)\n\n", argv[0], (unsigned int)led_data);
39
40 return 0;
41 }
42 U_BOOT_CMD(
43     led, 2, 0, do_KCCLED,
44     "led - kcci LED Test.",
45     "number - Input argument is only one(led [0x00-0xff])\n");
-----
~//u-boot/common$ vi common/Makefile
8 obj-y += cmd_kcci_led.o
-----
~//u-boot$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- all
- 응용예제 LED 불 제어 한 후 key 상태를 읽어 O, X 로 출력하기(key7 : 종료)
U-Boot> led 0x55
*LED TEST START
0:1:2:3:4:5:6:7
X:X:X:X:X:X:X:X
0:1:2:3:4:5:6:7
X:X:X:X:X:X:X:X
*LED TEST END(key : 0x80)

```

3.3 라즈베리파이 Kernel 빌드
 pi@pi00:~\$ uname -a //라즈베리파이 현재 커널 버전 확인


```

Linux pi00 6.1.0-rpi4-rpi-v8 #1 SMP PREEMPT Debian 1:6.1.54-1+rpi2 (2023-10-05) aarch64 GNU/Linux
- Ubuntu 호스트 크로스 컴파일
$ ubuntu@ubuntu00:~$ pwd
/home/ubuntu/pi_bsp
$ mkdir kernel ; cd kernel
$ sudo apt install git bc bison flex libssl-dev make libc6-dev libncurses5-dev
$ sudo apt install crossbuild-essential-armhf //32bit toolchain
$ git clone --depth=1 -b rpi-6.1.y https://github.com/raspberrypi/linux
$ cd linux ; pwd
/home/ubuntu/pi_bsp/kernel/linux
$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- bcm2711_defconfig
$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- zimage modules dtbs
-i4

- LCD 사용하기 위해 u-boot.bin을 사용 안함
u-boot$ vi /media/ubuntu/boots/config.txt //플 라인에 추가
-----
#kernel=u-boot.bin //u-boot 부팅 주소 처리
-----

- 커널 이미지 타겟 보드로 전송
라즈베리파이 SD카드 ubuntu 연결 후 (가장머신 usb3.0 활성화)
$ df
/dev/sdb1 261108 75820 185288 30% /media/ubuntu/boots
/dev/sdb2 30322460 15917604 13048344 55% /media/ubuntu/roots
~/kernel/linux$ sudo make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-INSTALL_MOD_PATH=/media/ubuntu/roots modules_install
~/kernel/linux$ sudo cp /media/ubuntu/boots/kernel71.img /media/ubuntu/boots/kernel71-backup.img
~/kernel/linux$ sudo cp arch/arm/boot/zimage /media/ubuntu/boots/kernel71.img
~/kernel/linux$ sudo cp arch/arm/boot/dts/*.dtb /media/ubuntu/boots/
~/kernel/linux$ sudo cp arch/arm/boot/dts/overlays/*.dtb* /media/ubuntu/boots/overlays/
~/kernel/linux$ sudo cp arch/arm/boot/dts/overlays/README /media/ubuntu/boots/overlays/

```

```

~/kernel/linux$ sudo vi /media/ubuntu/boots/config.txt //제일 아래 추가
-----
arm_64bit=0
-----
SD카드 라즈베리파이 연결 후 부팅
pi@pi00:~$ uname -a //커널 버전 및 빌드 시간
Linux pi00 6.1.66-v7l+ #1 SMP Mon Dec 11 11:57:35 KST 2023 armv7l GNU/Linux
- 리눅스 커널 ctags 설치
$ sudo apt install vim universal-ctags
$ ARCH=arm make tags
$ vi ~/vimrc
-----
set tags+=/home/ubuntu/kernel/linux/tags
-----
- 리눅스 커널 boot logo 이미지 변경
1.리눅스 gimp 설치 및 이미지 준비
$ pwd
/home/ubuntu/kernel
$ sudo apt install gimp
$ gimp //실행후 파일 --> 열기 --> 그림 선택 후 마우스 오른쪽버튼 메뉴 이미지 -->(변형 : 회전)
오른쪽버튼 메뉴 이미지 --> 이미지 크기 조정(연결고리해제) --> 800x480 --> 크기 조정
파일 --> Export As... --> 파일유형선택 --> ppm이미지 --> ascii 저장
2.변환 툴 설치
$ sudo apt install netpbm
3.커널 사용 이미지 변환
-----
ppm 800x480 이미지 준비후 (gimp)
$ pnmquant -fs 224 저장파일을 ppm > logo_kcci_224.ppm
$ pnmnoraw logo_kcci_224.ppm > logo_kcci_clut224.ppm
-----
4. //복사 및 메뉴 편집
$ cp log_kcci_clut224.ppm linux/drivers/video/logo/ //복사
$ cd linux/drivers/video/logo/
logo$ vi Kconfig //코드 추가
-----
71 config LOGO_KCCI_CLUT224
72 bool "kcci 224-color Linux logo"
73 depends on LOGO
74 default y

```

```

5. logo$ vi Makefile //코드 추가
-----
16 obj+$(CONFIG_LOGO_KCCL_CLUT224) += logo_kccl_clut224.o
-----
6. logo$ vi logo.c //코드 추가
-----
103 #ifdef CONFIG_LOGO_KCCL_CLUT224
104     logo = &logo_kccl_clut224;
105 #endif
-----
7. logo$ vi ./../include/linux/linux_logo.h //코드 추가
-----
48 extern const struct linux_logo logo_kccl_clut224;
-----
logo$ cd .././; pwd
/home/ubuntu/kernel/linux
8. make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- menuconfig // 커널 옵션 수정
Device Drivers ----> Graphics support ----> [*] Bootup logo
----> [*] Kccl 224-color Linux logo (NEW) //적당 후 종료
$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- zimage -j4

```

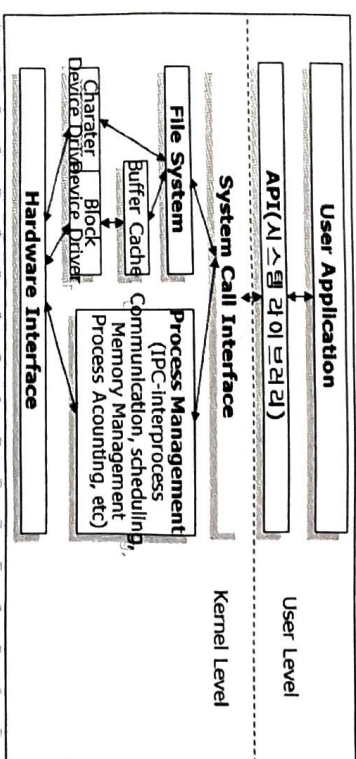
```

- 커널 이미지 타겟(pi) 보드로 전송
~/kernel/linux$ cp arch/arm/boot/zimage /srv/nfs/
- 타겟 보드 터미널에서 커널이미지 sd카드로 복사 및 재부팅
$ sudo cp /mnt/ubuntu_nfs/zimage /boot/firmware/kernel71.img
pi@pi00:~$ sudo reboot //부팅시 LCD 화면 확인 및 안료

```

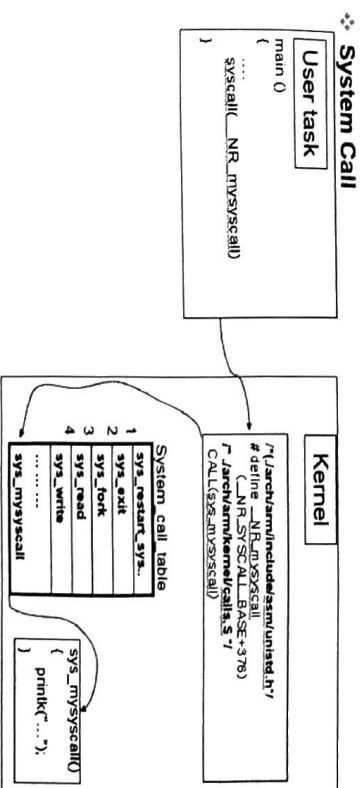
- 타겟 보드(ARM) 커널 시스템 콜 함수 구현

시스템 콜 함수란 user mode process와 kernel 간의 interface
 user mode process는 일반적으로 kernel 영역에 직접적으로 접근할 수 없다
 → kernel의 자료구조 및 hardware에 대한 접근 불가
 user mode process가 kernel이 가지고 있는 시스템의 상태 정보를 열람하거나 hardware에 접근하여 hardware를 통제하기 위해서는 kernel과의 communication channel이 필요.



-. POSIX API (Application Programming Interface)
 유닉스 운영체제에 기반을 두고 있는 일련의 표준 운영체제 인터페이스.
 application이 시스템에 각 서비스를 요청할 때에 어떠한 함수를 사용해야 하는지 지정한 것.
 표준을 두어 각각 다른 시스템에 응용 프로그램을 porting 하는 것이 용이 하게 하기 위한 목적. open(), close(), read(), write() 등

-. System Call
 소프트웨어 인터럽트를 통해 커널에 서비스를 요청하는 것
 Linux에서는 POSIX API를 준수하는 library 함수 안에서 system call 함수를 호출함으로써 system call을 사용한다.



시스템 콜 함수 추가하기

❖ 목적

❖ 기존 kernel에서 제공하지 않는 service를 user application에 제공
 → 새로운 System Call 을 만든다.

❖ 단계

- ❖ 커널 수정(Target board Kernel)
 - ❖ System Call 번호 할당 (/include/uapi/asm-generic/unistd.h)
 - ❖ System Call 호출 테이블 등록(/arch/arm/tools/syscall.tbl)
 - ❖ System Call 호출 함수 구현(/kernel/test_mysyscall.c)
 - ❖ Kernel 컴파일 및 보드퓨징, 재부팅
 - ❖ user application 제작
 - ❖ 새로운 System Call을 사용하는 application 제작 (syscall_app.c)
- \$ pwd
/home/ubuntu/pi_bsp/kernel/linux
1. System Call 번호, 및 함수 등록
- ❖ Linux 커널이 제공하는 모든 시스템 호출은 각각 고유한 번호를 가지고 있다.
 - ❖ linux\$ vi include/uapi/asm-generic/unistd.h에 mysyscall에 sys_mysyscall 함수와 번호를 등록한다
 - ❖ 다음과 같이 추가할 System Call에 고유번호와 call 함수명을 저장.

```

define __NR_mysyscall 451
__SYSCALL(__NR_mysyscall, sys_mysyscall)

#define __NR_syscalls
#define __NR_syscalls 452
889,1      943

```

- ❖ unistd.h 파일의 위 내용을 보면 현재 system call 은 450번까지 있는 것을 확인할 수 있다. 추가할 system call은 위와 같이 451번으로 할당한다.
- ❖ __NR은 System Call 고유번호를 나타내는 접두어이며 그 뒤의 mysyscall이 추가할 System Call 처리 함수의 이름이다.

2. System Call 테이블에 System Call 처리 함수 등록

- ❖ 각 함수들은 unistd.h에 정의 되어있는 system call 번호를 인덱스로 하여 접근된다. → ENTRY에 추가할 System Call 처리 함수 등록
 - ❖ linux\$ vi arch/arm/tools/syscall.tbl 파일을 vi로 연다.
- 다음과 같이 추가할 System Call 처리 함수를 등록한 후 저장.

```

449 common futex_wait          sys_futex_wait
450 common set_mempolicy_home_node sys_set_mempolicy_home_node
51 common mysyscall          sys_mysyscall

```

- ❖ linux\$ include/linux/syscalls.h 파일을 vi로 연다.

```

1387 int optlen);
1388 #asm(
1389     .long sys_mysyscall(long val);
1389     .endif

```

System Call 호출 함수 구현

- ❖ System Call 이 발생했을 때 수행될 함수를 구현한다.
- ❖ linux\$ vi kernel/test_mysyscall.c 를 다음과 같이 작성한다.

```

include <linux/kernel.h>

asm(
    .long sys_mysyscall(long val)
)
printk(KERN_INFO "Welcome to KCCI's Embedded System!! app value=%d\n", val);
return val*val;
}
"kernel/test_mysyscall.c" 7 lines, 172 characters

```

- ❖ linux\$ vi kernel/Makefile에 test_mysyscall.o 등록

```

12 notifier.o ksysfs.o cred.o reboot.o \
13 async.o range.o smboot.o ucount.o regset.o test_mysyscall.o
14
15 obj-$(CONFIG_USERMODE_DRIVER) += usermode_driver.o

```

커널, 시스템콜 함수 빌드

- ❖ linux\$ ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- make zImage

-i4

- ❖ 타겟 보드 퓨징 및 재 부팅

3. User App 작성하기(syscall_app.c) - 라즈베리파이에서 작성/빌드/실행
- ❖ pi@pi00:~\$ mkdir systemcall_test ; cd systemcall_test
 - ❖ pi@pi00:~/systemcall_test \$ vi syscall_app.c

```

1 #include <stdio.h>
2 #include <unistd.h>
3 #include <asm-generic/unistd.h>
4 #pragma GCC diagnostic ignored "-Wunused-result"
5 int main()
6 {
7     long val;
8     printf("Input value = ");
9     scanf("%ld", &val);
10    val = syscall(__NR_mysyscall, val);
11    if(val < 0)
12    {
13        perror("syscall");
14        return 1;
15    }
16    printf("mysyscall return value = %ld\n", val);
17    return 0;
18 }

```

- ❖ ubuntu에서 pi 커널에서 작성한 include/uapi/asm-generic/unistd.h 파일을 라즈베리파이 /usr/include/asm-generic/unistd.h로 복사 후 컴파일
- ❖ pi@pi00:~/systemcall_test\$ gcc syscall_app.c -o syscall_app
- ❖ pi@pi00:~/systemcall_test\$./syscall_app


```

COMB - P/TTY
pi@iot00:~/systemcall_test$ ./syscall_app
input value = 5
mysyscall return value = 25
pi@iot00:~/systemcall_test$

```

pi@pi00:~/systemcall_test\$:- \$ dmesg

[5.107541645] Welcome to KCCl's Embedded System!! app value=5

응용실습1) 기존 코드를 수정하여 LED제어를 시스템 콜함수로 구현하기 (파트로더 LED 상태 O, X 출력 예제 응용)
응용실습2) 시스템 콜 함수를 수정하여 KEY 값을 리턴하고 리턴된 값을 아크먼트로 전달하여 LED 제어

GPIO 커널 API

int gpio_request(unsigned gpio, const char *label);

void gpio_free(unsigned gpio);

int gpio_direction_input(unsigned gpio);

int gpio_direction_output(unsigned gpio, int value);

void gpio_set_value(unsigned gpio, int value);

int gpio_get_value(unsigned gpio);

int gpio_to_irq(unsigned gpio);

void free_irq(unsigned int irq, void* dev_id);

• 디바이스드라이버 커널 포함하기 (수업소스 drivers에 포함 예제 사용)

~ /linux\$ vi drivers/char/Kconfig

7 source "drivers/char/kccl_ledkey/Kconfig"

~ /linux\$ vi drivers/char/Makefile

5 obj-\$(CONFIG_KCCL_LEDKEY) += kccl_ledkey/

위의 디바이스드라이버 디렉토리 생성

~/linux\$ mkdir drivers/char/kccl_ledkey

기존 예제를 복사하여 이름 변경 및 수정하여 사용

\$cp -a p432_ledkey_poll_ledkey_builtin

디바이스드라이버 소스 p432_ledkey_dev.c 드라이버 파일을 ledkey_dev.c 복사하여 편집 (모듈 이름/디바이스파일 이름도 수정)

176 static __init int ledkey_mod_init(void)
185 static __exit void ledkey_mod_exit(void)

수정한 ledkey_dev.c를 커널 drivers/char/kccl_ledkey 디렉토리 복사
~ /linux\$ vi drivers/char/kccl_ledkey/Kconfig

1 menu "kccl gpio test device driver"
2 depends on ARM
3 config KCCL_LEDKEY
4 tristate "kccl ledkey device driver"
5 help
6 sample driver for ledkey device
7 endmenu

~ /linux \$ vi drivers/char/kccl_ledkey/Makefile

1 obj-\$(CONFIG_KCCL_LEDKEY) += ledkey_dev.o

~ /linux\$ ARCH=arm make menuconfig

→ Device Driver → Character driver →

kccl test device driver → kccl ledkey device driver 선택

1. 모듈<M> 로 빌드 후 타겟 보드에 ledkey_app 테스트

```
~ linux$ ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- make modules -j4
모듈로 컴파일하여 타겟 보드의 /lib/modules/6.1.53-v7l/kernel 디렉토리로 복사후
```

```
pi@pi00:/lib/modules/6.1.53-v7l$ pwd
/lib/modules/6.1.53-v7l
```

```
pi@pi00:/lib/modules/6.1.53-v7l $ sudo depmod -a
```

```
pi@pi00:/lib/modules/6.1.53-v7l $ cat modules.dep | grep ledkey
```

```
kernel/ledkey_dev.ko:
```

```
pi@pi00:/lib/modules/6.1.53-v7l $ sudo modprobe ledkey_dev
```

```
//application 실행 후 확인
```

```
[pi@pi00$ lsmod
```

Module	Size	Used by
ledkey_dev	3595	0

2. 빌트인<*> 로 빌드 후 타겟 보드에 ledkey_app 테스트

```
linux$ ARCH=arm make menuconfig
```

```
linux$ ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- make zimage -j4
```

```
커널 업그레이드 후 재 부팅 후 디바이스드라이버 확인 후 테스트
```

```
pi@pi00$ cat /proc/devices
```

```
linux$ cp .config arch/arm/configs/bcm2711_kcci_defconfig
```

```
필요시 make clean; make mrproper 후 커널 압축 배포
```

```
linux$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-
bcm2711_kcci_defconfig
```


-두트파일 시스템

1. Yocto Project - Embedded Linux 빌드시스템

Yocto Project - ubuntu20.04

```
~/Wroot$ mkdir yocto ; cd yocto //작업디렉토리
$pwd
/home/ubuntu/pi_bsp/roots/yocto
$ sudo apt update //필수 패키지 설치
$ sudo apt-get install gawk wget git diffstat unzip texinfo gcc-multilib build-essential chrpath socat lsbdl1.2-dev xterm python zstd libbz4-tool
-Release 4.0 (kirkstone)
$ git clone -b kirkstone git://git.yoctoproject.org/poky.git //poky 다운로드
$ cd poky
meta-raspberrypi 레이어 다운로드
poky$ git clone -b kirkstone git://git.yoctoproject.org/meta-raspberrypi
poky$ source oe-init-build-env //build 환경 적용
build$ vi conf/local.conf //미신 등록
-----
38 #MACHINE ??= "gemux86-64"
39 MACHINE ??= "raspberrypi4" //??=마지막 실경값, ?=처음실경값 사용
-----
build$ vi conf/bblayers.conf //레이어 경로 등록
-----
12 /home/ubuntu/pi_bsp/roots/yocto/poky/meta-raspberrypi W
-----
build$ bitbake core-image-minimal // 빌드시간 (real 157m34.394s)
-----
build$ cd tmp/deploy/images/raspberrypi4 //생성 이미지 위치
1.txt3 이미지로 sd카드 저장 방법
raspberrypi4$ ls *ext3
raspberrypi4$ sudo mount /dev/sdb? //SD카드 ubuntu연결후 장치파일 언마운트
raspberrypi4$ sudo dd if=core-image-minimal-raspberrypi4.ext3 of=/dev/sdb2 bs=1M status=progress
2. bz2 압축파일로 sd카드 저장 방법
```

sd카드 ubuntu 마운트 후 기존 내용 삭제(lost+found 파일유지) 또는 포맷 후 압축 해제

```
raspberrypi4$ sudo mount /dev/sdb2 //마운트 해제
raspberrypi4$ sudo mkfs.ext3 /dev/sdb2 //ext3 포맷
raspberrypi4$ sudo e2label /dev/sdb2 roots //라벨명 설정
raspberrypi4$ sudo tar xvf rpilinux-image-raspberrypi4.tar.bz2 -C /media/ubuntu/roots //sd카드 마운트 후 압축 풀기
부팅후 login: root , passwd : 없음
```

-사용자정의 레이어/패시과 추가 및 빌드(ssh, python)

```
$ pwd
/home/ubuntu/pi_bsp/roots/yocto/poky
poky$ mkdir meta-rpilinux ; cd meta-rpilinux ; mkdir conf ; cd conf
conf$ vi layer.conf
-----
```

```
# We have a conf and classes directory, add to BBPATH
BBPATH .= ":${LAYERDIR}"
```

```
# We have recipes-* directories, add to BBFILES
BBFILES += "${LAYERDIR}/recipes-*/*/*.bb $
${LAYERDIR}/recipes-*/*/*.bbappend"
```

```
BBFILE_COLLECTIONS += "rpilinux"
BBFILE_PATTERN_rpilinux = "${LAYERDIR}"
BBFILE_PRIORITY_rpilinux = "0"
LAYERSERIES_COMPAT_rpilinux = "kirkstone"
```

```
conf$ cd ..
meta-rpilinux$ mkdir recipes-rpilinux ; cd recipes-rpilinux ; mkdir images ; cd images
images$ vi rpilinux-image.bb //패키지 등록
-----
IMAGE_INSTALL += "libstdc++ + mtd-utils"
IMAGE_INSTALL += "openssh openssl"
```

```
$pwd
~/pi_bsp/roots/yocto/poky/build/conf$ //추가된 레이어 경로 등록
conf$ vi bblayers.conf
-----
```

```
13 /home/ubuntu/pi_bsp/roots/yocto/poky/meta-rpilinux
```

```
build$ bitbake -s core-image-minimal | grep openssh //다운로드된 패키지 확인
openssh :8.9p1-r0
build$ bitbake -c cleanall core-image-minimal //image delete
```

```

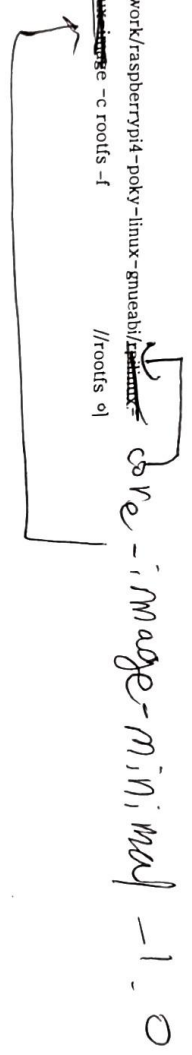
build$ bitbake rpilinux-image
build$ cd tmp/deplo/images/raspberrypi4/
raspberrypi4$ sudo dd if=rpilinux-image-raspberrypi4.ext3 of=/dev/sdb2 bs=1M
status=progress
SD 카드 부팅 후 확인 ssh 접속 확인

```

```

압축 전 파일 시스템 위치
build$ cd poky/build/tmp/work/raspberrypi4-poky-linux-gnueabi/
image-1.0-r0 bitbake rpilinux-image -c roots -f //roots 이
이미지 생성
roots$ du -h -d 1
대표적인 recipe 종류

```



```

poky$ bitbake core-image-minimal-dev //개발에 쓰이는 헤더와 라이브러리 포함
poky$ bitbake core-image-full-cmdline //운송가능한 리눅스시스템 대부분 포함
poky$ bitbake core-image-x11 //터미널을 제공하는 기본적인 x11 이미지
poky$ bitbake core-image-sato //GNOME이 포함된 이미지 생성

```

```

yocto zimage
pi@pi00:~$ uname -a
Linux pi00 5.15.92-v7l #1 SMP Wed Feb 8 16:47:50 UTC 2023 armv7l GNU/Linux

```